

Taller 03 v2: solver multiagente con GraphRAG y H200

Autor: Alejandro Chavez. Entrega preparada el 3 de octubre de 2026.

Este trabajo construye un sistema que recibe tareas en PDF, recupera restricciones y teoría, propone un plan, escribe Python, lo ejecuta aisladamente, critica y produce el entregable. Se implementan ocho responsabilidades con un bucle propio y se evalúa frente a una ablación y a una memoria de scripts aprobados.

Procedencia del material. Por petición del usuario se reconstruyó el kit solver-v2 a partir del PDF vigente y los archivos disponibles. Contiene tres enunciados de práctica propios, dos controles reales aportados y notas teóricas del curso. No se presenta como una copia del paquete institucional. Los identificadores demo-A/B/C nombran las tres prácticas propias del kit. Se incluyen 25 checks centrales de práctica, 9 diagnósticos adicionales y 21 checks de los controles. Las respuestas esperadas se calculan con código independiente.

La inferencia y los embeddings de las corridas finales se hicieron en la H200 de la universidad. Codex editó y verificó los archivos como orquestador; no se generaron subagentes Codex para estas corridas. El código científico se ejecutó localmente en macOS. Se generaron exactamente dos ilustraciones mediante Codex, con sus prompts; las gráficas y diagramas se construyeron mediante código. Se conservan los fallos medidos: completar la entrega del taller no significa que un solver haya aprobado todos los checks.

Parte 0. Tres fallas que pueden pasar inadvertidas

0.a. Un reporte sin ejecución

Se hizo una única llamada H200 con el enunciado reconstruido A y sin herramientas. El modelo reconoció que no podía medir las exactitudes y dejó vacía la lista de estimaciones. Por ello **no se reprodujo la invención de exactitudes** observada por el profesor: cambiaron el enunciado, el modelo servido y el prompt. La salida completa está en solver-v2/parte0/a_sin_ejecutar.json, junto con ruta, fecha, consumo y el contraste calculado posteriormente.

Que una cifra esté cerca de la real no la hace aceptable: falta la ejecución que vincula datos, partición, script y resultado. Aquí no hubo una cifra inventada de exactitud que cambiara la conclusión; asignar a esta corrida el cruce de curvas del ejemplo del PDF sería falsear la observación. Para detectar el problema sin conocer la respuesta, el verificador debe exigir un artefacto ejecutado y comprobar que cada cifra del reporte procede de él o del enunciado.

La respuesta sin herramientas sí muestra un límite: anuncia curva.png aunque el modelo no creó el archivo. También calcula el tamaño de prueba usando round; el protocolo general de scikit-learn usa el redondeo requerido por su API y no debe deducirse de ese ejemplo. Su tabla deja las exactitudes por medir. Una explicación plausible con código propuesto sigue sin ser un experimento.

Salida observada: «**Nota de transparencia:** no tengo herramientas de ejecución, por lo que **no puedo medir** accuracy_nb, accuracy_lr ni los valores de la curva de aprendizaje. Este reporte entrega el protocolo completo, el código reproducible y la discusión; los valores medidos deben completarse ejecutando el código. No invento cifras ni citas.»

Métrica	LLM sin herramientas	Referencia ejecutada
accuracy_nb	por medir	0.9386
accuracy_lr	por medir	0.9825
accuracy_lr_f005	por medir	0.9386
accuracy_lr_f020	por medir	0.9737
accuracy_lr_f100	por medir	0.9825

0.b. Recuperación plana frente a dependencias

El script TF-IDF top-2 recuperó las Partes 4 y 3 del nuevo enunciado A. Ninguna contenía los tres hechos del protocolo: dataset, tamaño de prueba y estratificación. La regla de referencias siguió «Parte 1» y el contexto pasó de **0 a 3 hechos**. El proceso es local y no llama a ningún modelo.

La falla nace en la recuperación: antes de generar, el contexto ya perdió una restricción necesaria. Un LLM podría completar el vacío con una división habitual o con otro dataset y producir código válido para una tarea distinta. La arista se extrae con una regla porque la referencia es una obligación literal, reproducible y auditable; las relaciones semánticas del LLM se añaden encima de ese esqueleto.

0.c. Código de salida cero con fuga

Un árbol de decisión entrenado y evaluado con el mismo X produjo accuracy=1.0, escribió JSON y terminó con código cero. Un evaluador ingenuo lo aceptó. La guarda AST detectó la misma expresión en ajuste y evaluación; la plausibilidad numérica añadió una alerta. Para ejecutar este ejemplo deliberadamente incorrecto se omitió únicamente esa guarda sobre un script fijo confiable; se conservó el aislamiento del sistema operativo.

La heurística no detecta todas las fugas: ajustar un escalador con todo el dataset antes de dividir puede pasar inadvertido. Las comprobaciones por código deben ir antes del crítico; el LLM revisa después el sentido del protocolo

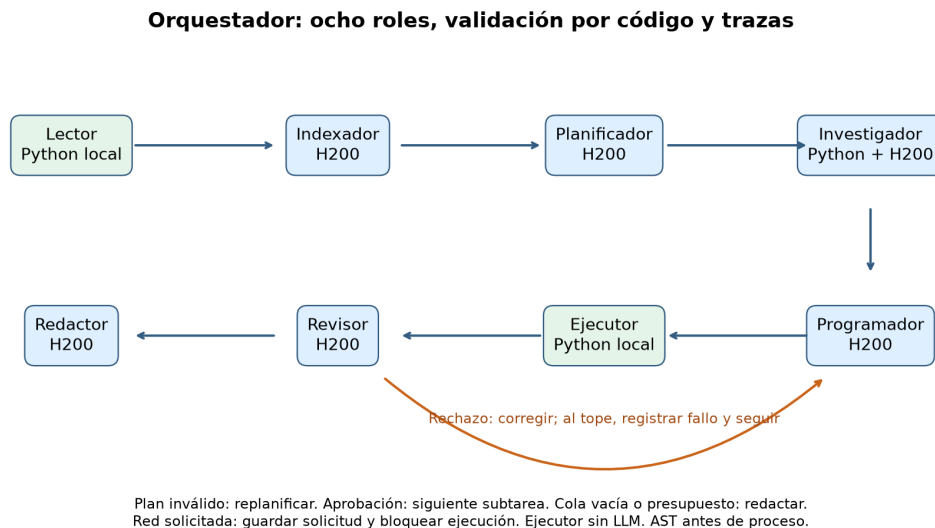
y sus condiciones. Una exactitud perfecta es una alerta, no una prueba automática de fuga.

Parte 1. Arquitectura, grafo y controles

Ocho responsabilidades y aristas condicionales

El lector extrae páginas, tablas y ligaduras y detecta páginas que requieren OCR. El indexador crea secciones, dependencias, entidades normalizadas y comunidades. El planificador propone subtareas, criterios y dependencias. El investigador recupera contexto citado. El programador propone un script. El ejecutor es Python, sin LLM. El crítico revisa después de las comprobaciones genéricas. El redactor responde las preguntas y produce el formato solicitado.

El bucle permite replanificar si el DAG es inválido; devolver una corrección al programador si ejecución o crítica rechazan; avanzar por aprobación; registrar fallo al alcanzar el tope y continuar tareas independientes; bloquear red hasta decisión humana; y redactar al terminar la cola o al agotarse el presupuesto. Este bucle propio es una alternativa permitida a LangGraph.



Orquestador: roles, correcciones y condiciones de salida.

Las seis correcciones

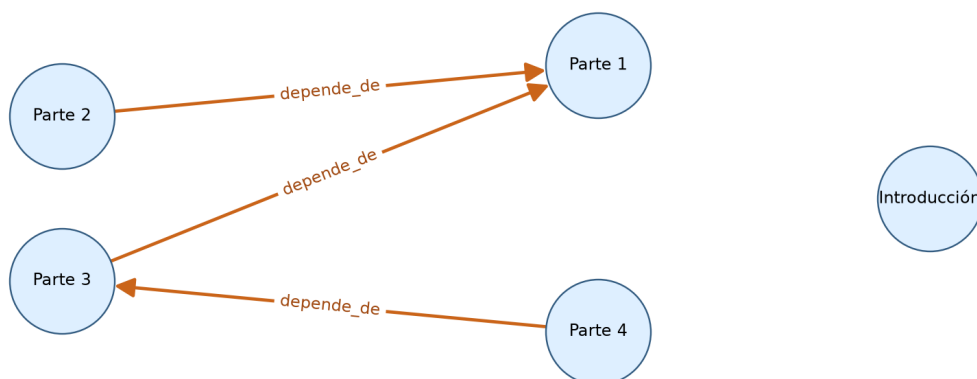
Corrección obligatoria	Código responsable	Qué verifica
Plan como DAG	<code>validate_plan</code>	IDs, referencias existentes, ausencia de ciclos y cobertura formal de secciones.
Esqueleto GraphRAG	<code>build_sections</code> , <code>dependency_closure</code> , <code>KnowledgeGraph</code>	Referencias por reglas, cierre literal, embeddings BGE-M3, entidades y relaciones H200 normalizadas, comunidades Louvain resumidas.
Sandbox	<code>guard_script</code> , <code>execute_script</code>	AST antes del proceso, cwd propio, entorno vacío, Seatbelt, timeout y killpg, red bloqueada.
Resultados antes del crítico	<code>validate_results</code>	JSON, cifras finitas, salida del proceso, artefactos y señales heurísticas de fuga.
Procedencia antes de publicar	<code>render_provenance</code>	Sustituye placeholders y rechaza cifras ajenas a ejecución o enunciado; conserva ledger.
Trazas en todos los caminos	<code>Trace</code> , <code>LocalTrace</code> , bloque <code>finally</code>	Peticiones, respuestas, modelo, uso, latencia, errores, decisiones y resultados de procesos.

Estas capas tienen límites concretos: cobertura formal no equivale a interpretar bien una sección, AST no prueba seguridad general y una cifra con procedencia puede estar descrita incorrectamente. Los estados del solver y el golden se reportan por separado.

Grafo del enunciado y teoría completa disponible

El enunciado A tiene cuatro partes. La curva remite al protocolo de la Parte 1; la discusión remite a la curva. Seguir estas aristas recupera condiciones que la similitud puede perder. El diagrama siguiente sale de grafo.json, no de relaciones inventadas para ilustrar el sistema.

Grafo del enunciado propio demo-A · conexiones extraídas por reglas



Dependencias determinísticas del enunciado A reconstruido.

El kit conserva el texto completo de las notas teóricas disponibles en capítulos numerados del curso, excluyendo índices, prácticas, talleres y recursos visuales. El inventario guarda sus hashes. Los fragmentos se vectorizan completos; la extracción de entidades H200 se hizo sobre cada documento completo. La preparación común queda en solver-v2/indice_curso/ y se reutiliza por hash en todas las variantes. Los resúmenes de comunidades usan extractos limitados de cada documento; orientan la búsqueda global y no reemplazan las fuentes literales. Las imágenes incrustadas en notas no se interpretaron mediante OCR.

La recuperación completa entrega siempre la sección y su cierre de dependencias, añade semillas por embeddings y resúmenes de comunidades pertinentes. La ablación entrega solamente los cuatro fragmentos más similares por embeddings: conserva la preparación para comparar recuperación con la misma base y el mismo modelo, pero elimina cierre, vecindad y búsqueda global. Los costes comunes de preparar la base se presentan aparte.

Documentos teóricos	Fragmentos	Comunidades	Tokens entrada comunes	Tokens salida comunes
103	1056	26	280902	82177

Modelo medido: zai-org/GLM-5.3-Flash. Endpoints: <http://172.28.230.10:12555/v1> y <http://172.28.230.10:12559/v1>. Verificación UTC: 2026-10-03T20:59:16.412857+00:00. Embeddings: bge-m3:latest, <http://172.28.230.10:11434/api/embed>.

Evidencia de rechazo H200

Una prueba adicional sembró el error softmax(logits * T). Pasó los checks genéricos de ejecución, pero el crítico H200 lo rechazó porque debe dividirse por temperatura. El programador H200 reparó el script, se ejecutó nuevamente y el crítico lo aprobó. Se conservan ambos scripts y las decisiones completas en solver-v2/critico_forzado/. Es un fallo forzado explícito, separado de las corridas del golden.

Parte 2. Golden independiente, ablación y fallos

Diseño del verificador reconstruido

Los tres ejercicios de práctica cubren NB frente a regresión logística y una curva de aprendizaje; temperatura, top-p, entropía y muestreo; y TF-IDF frente a LSA con datos en una tabla PDF. Los controles reales añaden un benchmark lista/diccionario y diferencias centrales con predicción previa. Hay entregables Markdown, notebook ejecutado y PDF de hasta dos páginas.

El golden tiene 25 checks centrales de práctica y 9 diagnósticos adicionales, más los 21 de controles reales. Cada control contiene al menos dos cifra con verdad_py, checks de formato y procedencia. Dos de los checks reales son diagnósticos semánticos añadidos después de revisar salidas narrativas: detectan que los candidatos no coinciden en el punto indicado. Su carácter post hoc está marcado en el JSON; no son una comparación preregistrada. Se mantienen todos los checks en la tabla detallada; no se eliminan los que fallan. El evaluador recalcula las referencias desde funciones conocidas, sin ejecutar código del LLM ni instrucciones arbitrarias del JSON.

La trampa de A exige usar únicamente train al ajustar el escalador y mantener el test para todos los tamaños. La de B exige incluir el token que cruza el umbral del núcleo. Control 3 requiere predicción antes del primer proceso y conservar en CSV los ceros numéricos. Control 1 tiene un oracle de cantidades y protocolo; sus tiempos no tienen una respuesta fija, porque dependen del estado del equipo. Las restricciones estáticas se declaran heurísticas y no

constituyen un analizador universal.

Comparación por tarea

Se hicieron nuevas corridas con el kit reconstruido y el corpus completo disponible. Cada pareja usa el mismo PDF, modelo, golden y base. El presupuesto, timeout y top-k permanecen constantes. Los planes y reintentos pueden variar: una sola corrida por tarea y variante no demuestra causalidad.

La recuperación de tablas se corrigió antes de fijar el conjunto final; se declara esta intervención en la implementación. Las tablas y el análisis utilizan exclusivamente las 15 corridas finales identificadas por tarea y variante. Los tokens se suman desde todas las llamadas LLM de la traza seleccionada, incluidos errores y reintentos. La preparación común de teoría se cuenta una sola vez y no se atribuye ficticiamente a cada tarea. Los embeddings se registran por separado. duracion_s suma latencias de llamadas y procesos; pared_s es la duración real registrada por tarea. El runner ejecuta dos tareas completas a la vez, mientras las subtareas de cada una siguen siendo secuenciales; no se eligió la extensión de paralelismo de subtareas.

Variante	Tarea	Estado	Intentos	Entrada	Salida	Pared (s)
completo	demo-A	completado	3	53585	13558	144.8400
completo	demo-B	completado	2	46933	13840	134.2975
completo	demo-C	completado	2	47576	11271	117.8554
completo	control-1	parcial	2	42527	8611	87.0468
completo	control-3	completado	4	68445	16493	128.6901
sin_grafo	demo-A	completado	3	53070	15383	128.4038
sin_grafo	demo-B	completado	2	38332	12839	98.6886
sin_grafo	demo-C	parcial	3	44912	17563	129.2439
sin_grafo	control-1	completado	1	29119	8288	72.0241
sin_grafo	control-3	completado	5	41310	13148	78.0581
memoria	demo-A	parcial	3	51259	11433	136.8706
memoria	demo-B	completado	2	39199	8444	90.9341
memoria	demo-C	completado	2	47629	14639	161.1928
memoria	control-1	completado	1	38962	7286	70.8071
memoria	control-3	parcial	3	114781	18966	170.6956

Checks de las mismas corridas:

Variante	Tarea	Pasan	Total	Centrales pasan	Centrales total	Procedencia
completo	control-1	8	8	8	8	1.0000
completo	control-3	11	13	11	11	1.0000
completo	demo-A	13	13	8	8	1.0000
completo	demo-B	12	12	9	9	1.0000
completo	demo-C	9	9	8	8	1.0000
memoria	control-1	8	8	8	8	1.0000
memoria	control-3	10	13	10	11	1.0000
memoria	demo-A	12	13	7	8	1.0000
memoria	demo-B	12	12	9	9	1.0000
memoria	demo-C	9	9	8	8	1.0000
sin_grafo	control-1	8	8	8	8	1.0000
sin_grafo	control-3	11	13	9	11	1.0000
sin_grafo	demo-A	13	13	8	8	1.0000
sin_grafo	demo-B	12	12	9	9	1.0000
sin_grafo	demo-C	3	9	3	8	0.0000

Comparación agregada (preparación común excluida y declarada arriba):

Variante	Pasan	Total	Centrales pasan	Centrales total	Entrada	Salida	Intentos
completo	53	55	44	44	259066	63773	13
memoria	51	55	42	44	291830	60768	11
sin_grafo	47	55	37	44	206743	67221	14

Consumo por responsabilidad; el CSV conserva también la desagregación por tarea:

Variante	Rol	Entrada	Salida	Llamadas
completo	critico	63709	2332	13
completo	indexador	106695	35752	79
completo	lector_prediccion	762	359	1
completo	planificador	3872	3076	5
completo	programador	47796	12885	13
completo	redactor	36232	9369	5
memoria	critico	52686	1442	11
memoria	indexador	84041	29801	54
memoria	lector_prediccion	762	237	1
memoria	planificador	3872	2960	5
memoria	programador	64891	10978	11
memoria	redactor	85578	15350	8
sin_grafo	critico	34088	1932	12
sin_grafo	indexador	92266	31308	61
sin_grafo	lector_prediccion	762	241	1
sin_grafo	planificador	3872	3053	5
sin_grafo	programador	29325	13477	14
sin_grafo	redactor	46430	17210	10

Cómo interpretar las cifras

Una accuracy mide aciertos sobre un conjunto concreto; no demuestra que una familia de modelos sea universalmente mejor. El pipeline evita que el escalador aprenda de test. La curva mantiene un test fijo mientras cambia la cantidad de train; con una semilla única su forma puede fluctuar.

La temperatura transforma logits antes de softmax: temperatura menor concentra probabilidad. Top-p conserva el prefijo mínimo cuya masa alcanza el umbral y después puede renormalizar para muestrear. En este ejercicio el muestreo pedido usa la distribución completa; mezclarlo con el núcleo cambiaría la respuesta correcta.

TF-IDF utiliza coincidencias ponderadas de palabras. LSA proyecta sobre un espacio de menor dimensión; puede conectar términos, pero también perder temas en un corpus artificial muy pequeño. El resultado depende de la dimensión, normalización y desempate. Precision@2 divide el número de relevantes entre dos; no mide todas las posiciones del ranking.

El benchmark no incluye la construcción dentro del reloj. La lista busca secuencialmente; el diccionario usa hashing con acceso promedio constante bajo condiciones habituales. Las diferencias centrales pueden fallar al disminuir demasiado el paso: las perturbaciones llegan a almacenarse como el mismo número de punto flotante y la resta pierde información. La comparación en un punto no demuestra una derivada en todo el dominio.

Tres comprobaciones peores y atribución por traza

Se seleccionan fallos del baseline y de la ablación, priorizando cifras, restricciones e integridad antes de formato. En cada uno se muestra la comprobación, el contexto que recibió el programador, lo producido y la decisión del código o del crítico. La responsabilidad se asigna a una etapa verificable; no se deduce de la fluidez de la explicación. Los JSON y scripts completos quedan en el directorio de la corrida.

control-3-igualdad_w2 (completo). Apunta a programador y crítico. El script afirma igualdad de candidatos en el punto indicado, pero el oracle calculado la contradice. La crítica genérica aprobó el script sin detectar esa afirmación conceptual falsa. El evaluador registró: Posición w=2.0: A=10.0; B=4.0; igualdad calculada=False; afirmación

detectada=True. Diagnóstico post hoc de salida o texto del script: en coincidir en un punto (de hecho $g_A(2) = g_B(2) = 10$). Para acercarse a una conclusion mas gen. Se localizó la evidencia en la subtask t5, con 1 llamadas al programador y 1 decisiones del crítico. Su última decisión fue approved=True. El notebook muestra el contexto relevante, el fragmento responsable y la decisión; la traza conserva los mensajes íntegros. Evidencia: corridas/tarea-control-3-completo-kit/traza.jsonl, resultados_verificados.json y el script en subtasks/t5/.

control-3-igualdad_w_tercio (completo). Apunta a programador y crítico. El script afirma igualdad de candidatos en el punto indicado, pero el oracle calculado la contradice. La crítica genérica aprobó el script sin detectar esa afirmación conceptual falsa. El evaluador registró: Posición $w=0.3333333333333333$: $A=-1.6666666666666667$; $B=-1.0$; igualdad calculada=False; afirmación detectada=True. Diagnóstico post hoc de salida o texto del script: unto y divergir en otros (de hecho g_A y g_B coinciden en $w = 1$ y $w = 1/3$). " "Otra comprobación sin program. Se localizó la evidencia en la subtask t3, con 2 llamadas al programador y 2 decisiones del crítico. Su última decisión fue approved=True. El notebook muestra el contexto relevante, el fragmento responsable y la decisión; la traza conserva los mensajes íntegros. Evidencia: corridas/tarea-control-3-completo-kit/traza.jsonl, resultados_verificados.json y el script en subtasks/t3/.

control-3-csv_integridad (sin grafo). Apunta a programador y crítico. Una fila del CSV perdió un cero válido y quedó vacía. Los resultados JSON correctos no bastan para aprobar la integridad del archivo tabular. El evaluador registró: Todos los CSV conservan tres pasos y ceros numéricos; se aceptan alias explícitos con el mismo significado.. Se localizó la evidencia en la subtask t2, con 1 llamadas al programador y 1 decisiones del crítico. Su última decisión fue approved=True. El notebook muestra el contexto relevante, el fragmento responsable y la decisión; la traza conserva los mensajes íntegros. Evidencia: corridas/tarea-control-3-sin_grafo-kit/traza.jsonl, resultados_verificados.json y el script en subtasks/t2/.

Parte 3. Cuatro frenos forzados

Se forzó cada freno con un cliente de guion, sin inferencia H200. Presupuesto: se verifica antes de pedir y reserva al redactor; el trabajo no realizado queda explícito en una entrega parcial. Intentos y repetición: el mismo hash rechazado no vuelve a ejecutarse, la subtask falla al tope y sigue una independiente. Timeout: el ejecutor mata el grupo de procesos del bucle infinito. Red: una descarga crea solicitud_red.json ligada a hashes y queda bloqueada antes del proceso hasta aprobación humana.

La descarga bloqueada no se realizó ni se simuló una aprobación. La reanudación con un checkpoint es la opción D del taller; aquí se eligió C. La evidencia requerida del freno es bloquear antes de salir a red, y está registrada. La prueba de timeout utiliza un límite de 1,5 segundos que permite arrancar el intérprete local y registra la continuación de la tarea independiente.

Freno	Estado	Procesos	Red solicitada	Red ejecutada	Entrega guardada	Llamadas H200
presupuesto	parcial	0	False	False	True	0
repeticion	parcial	2	False	False	True	0
timeout	parcial	2	False	False	True	0
red	parcial	1	True	False	True	0

Parte 4. Una extensión medida: memoria de scripts aprobados

Se eligió la opción C después de medir el baseline. Los scripts aprobados se guardan como nodos con hash, tarea, resultados, dataset fuente y cita de traza. Se enlazan con métodos y datasets; las tareas siguientes recuperan ejemplos por similitud de descripción. La memoria no concede aprobación: cada script propuesto atraviesa otra vez la guarda, ejecución y crítica.

Las cinco tareas se ejecutaron también con memoria y con el mismo golden. Se distingue coste de preparar la teoría, consumo por corrida e intentos de código. La memoria se sembró con scripts del baseline: mide reutilización de ejemplos de estos casos, no generalización a casos desconocidos. Solo se guardan scripts que llegaron a aprobación del crítico; esa aprobación puede ser errónea, como muestran los fallos. La comparación agregada y por tarea está en las tablas anteriores. Algunas salidas y textos de scripts aprobados contienen afirmaciones falsas de igualdad entre candidatos. Se conservan como evidencia y se señalan con checks semánticos; el informe final no las adopta como explicación correcta.

Parte 5. Reflexión basada en las mediciones

Las respuestas se redactaron mediante H200 a partir de los CSV finales. Se declara la asistencia y se comprueba que cada respuesta tenga entre 100 y 150 palabras.

Dónde vive el objetivo

El objetivo vive en el pipeline de resolución: Solver.solve orquesta el flujo, validate_plan verifica que el plan cumpla condiciones antes de ejecutar, y render_provenance produce la evidencia trazable del resultado. El estado completado no es decorativo: exige writer_valid (el escrito pasó sus validaciones), format_valid (el formato es correcto) y que todas las subtareas estén aprobadas; si falta cualquiera, el estado no puede ser completado. Conviene distinguir dos niveles de aprobación: la golden externa, independiente, que decide los requisitos medidos como checks y accuracy, y la aprobación académica del profesor o persona evaluadora, que es un juicio distinto. Un error común es confundir aprobación genérica con verdad científica: aprobar checks demuestra cumplimiento de criterios definidos, no validez universal de las conclusiones.

Qué compró el grafo y quién gastó más

Excluyendo la preparación común, el rol de mayor consumo es el indexador: 142447 tokens en la variante completo, 123574 en sin_grafo y 113842 en memoria. Este alcance corresponde a la fase de indexación de documentos, no a la escritura ni a la crítica. Al comparar completo contra sin_grafo, las cinco tareas completas gastaron 259066 tokens de entrada y 63773 de salida, con 13 intentos de código; sin_grafo gastó 206743 de entrada y 67221 de salida, con 14 intentos. En checks, completo aprobó 53 de 55 (44 de 44 centrales) y sin_grafo 47 de 55 (37 de 44 centrales). La preparación común (280902 tokens de entrada, 82177 de salida, 130 llamadas LLM) se amortiza aparte y no debe sumarse al consumo por rol. Como hubo una sola corrida por tarea y los planes varían entre corridas, estos números describen lo observado: no permiten afirmar causalidad ni ahorro de velocidad entre variantes.

Uso honesto y daño

Para un uso honesto hay que declarar que el kit fue reconstruido por solicitud, que la asistencia vino de H200 y Codex (distintas del trabajo del estudiante), y adjuntar scripts, PDF, CSV y trazas como evidencia. El caso de falla con exit0 y accuracy1 en presencia de una fuga demuestra que la ejecución exitosa por sí sola no basta: el código puede terminar bien y aun así hacer daño. Los cuatro frenos son: presupuesto con reserva, control de intentos y repetición, timeout con killpg para matar el grupo de procesos, y revisión humana antes de cualquier acceso a red. Los daños posibles si el sandbox falla incluyen leer archivos, exfiltrar datos, borrar contenido o agotar recursos. Conviene no sobreactuar: el análisis AST no prueba seguridad, y la reanudación con red no fue probada; ambas limitaciones deben declararse como incertidumbre, no como garantías.

Ilustraciones y declaración de asistencia

Se generaron exactamente dos ilustraciones mediante Codex. No se atribuyen imágenes a Nano Banana: no se obtuvo un PNG verificable de ese servicio. Los prompts están en evidencia/prompts_imagenes_codex.json. Los diagramas y gráficas científicas son figuras hechas por código y se cuentan aparte.

El estudiante debe comprender y asumir el contenido: los roles H200 propusieron planes, código y redacción; Codex implementó el orquestador, organizó archivos y comprobó la entrega. Los controles PDF fueron aportados por el usuario; no se inventa una autorización docente adicional. El material y las trazas se entregan para evaluar lo que realmente ocurrió.

Reproducibilidad y archivos de entrega

El notebook permite estudiar y recalcular referencias sin VPN. Se entrega ejecutado, con salidas y figuras. Nuevas inferencias necesitan acceso a la red universitaria. El sandbox de esta implementación necesita macOS y sandbox-exec; si no está disponible, el ejecutor bloquea scripts en vez de ejecutarlos sin aislamiento.

Instala Python 3.12 y requirements.txt, registra un kernel taller3-h200 y abre este notebook desde la carpeta extraída. Solver().solve(pdf, salida) expone el contrato del taller; .run(pdf) añade answer para el envoltorio siguiente. El kit incluye evaluar_solver.py, golden comentado, scripts de Parte 0, corpus, caché común, enunciados y manifiesto. La evaluación offline lee resultados; --ejecutar permite generar corridas faltantes mediante H200. Para repetir desde cero utiliza una salida nueva y no sobrescribas el historial medido.

corridas/tarea-*kit/ contiene directorios reales con plan, traza, código, resultados y entregables. El ZIP reúne notebook, informe PDF y fuente, código, entorno, golden, CSV, teoría, corridas y evidencia auxiliar. No incluye el entorno virtual ni claves. Las rutas históricas permanecen en trazas; el lector de artefactos del notebook resuelve las copias de la entrega al cambiar de carpeta.

Esta entrega desarrolla las Partes 0-5 del Taller 03 v2 con el kit reconstruido, evaluación independiente y resultados verificables, incluidos los fallos que se analizan. El conjunto experimental consta exclusivamente de las 15 corridas finales de cinco tareas y tres variantes.

Fuentes principales: taller-03-v2-solver-multiagente.pdf, los cinco enunciados, notas del curso con hashes, CSV crudos, trazas y evidencia/analisis_fallos_kit.json. Fundamentos técnicos documentados en las notas: prevención de fuga, softmax y top-p, recuperación léxica/LSA y validación de agentes. Ninguna cifra de evaluación procede de una estimación del redactor.